JobFinding: A Specialized Search Engine with Inverted Index

Sazzad Hossain - 0432220005101103

Suraia Akter Mim - 0432220005101108

A. O. M. Ramim Chowdhury - 0432220005101146

CELL 1: Install Kaggle and download the dataset directly

```
!pip install -q kaggle
!kaggle datasets download -d arshkon/linkedin-job-postings
!unzip -q -o linkedin-job-postings.zip -d linkedin_data
```

```
Dataset URL: https://www.kaggle.com/datasets/arshkon/linkedin-job-postings
License(s): CC-BY-SA-4.0
Downloading linkedin-job-postings.zip to /content
100% 159M/159M [00:09<00:00, 16.8MB/s]
```

CELL 2: Import Libraries

```
import pandas as pd
import numpy as np
import math
import re
from collections import defaultdict, Counter
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

# Download necessary NLTK data for preprocessing
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('punkt_tab') # Required for newer NLTK versions

stop_words = set(stopwords.words('english'))
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
```

CELL 3: Preprocessing Function

```
def preprocess_text(text):
    if pd.isna(text):
        return []

    # Lowercase the text
    text = str(text).lower()

    # Remove punctuation using regex (keep only alphanumeric and spaces)
    text = re.sub(r'[^a-z0-9\s]', '', text)

    # Tokenize
    tokens = word_tokenize(text)

    # Remove stop words
    filtered_tokens = [word for word in tokens if word not in stop_words]

    return filtered_tokens
```

CELL 4: Load Data and Build Inverted Index

```
print("Loading dataset...")
# The Kaggle dataset 'arshkon/linkedin-job-postings' contains 'postings.csv'
try:
    df = pd.read_csv('linkedin_data/postings.csv', nrows=5000)
except FileNotFoundError:
    # Fallback in case it unzipped directly to the root
    df = pd.read_csv('postings.csv', nrows=5000)
```

```
# Fill missing titles and descriptions
df['title'] = df['title'].fillna('Unknown Title')
df['description'] = df['description'].fillna('')

# Initialize our Inverted Index and Document Frequency dictionaries
inverted_index = defaultdict(lambda: defaultdict(int))
document_frequency = defaultdict(int)

Total_Documents = len(df)

print("Building Inverted Index... This may take a minute.")
for index, row in df.iterrows():
    # The ID column in this dataset is 'job_id'
    doc_id = row['job_id']
    text = f"{row['title']} {row['description']}"

    # Preprocess text
    tokens = preprocess_text(text)

    # Calculate Term Frequencies for this document
    term_counts = Counter(tokens)

    # Populate the Inverted Index
    for term, count in term_counts.items():
        inverted_index[term][doc_id] = count
        document_frequency[term] += 1

print(f"Inverted Index built successfully! Unique terms indexed: {len(inverted_index)}")
```

```
Loading dataset...
Building Inverted Index... This may take a minute.
Inverted Index built successfully! Unique terms indexed: 77993
```

CELL 5: Query Processing (Boolean Retrieval)

```
def get_boolean_results(query):
    # Determine the operator (default is OR if not specified)
    operator = "OR"
    if " AND " in query:
        operator = "AND"
        query_terms = query.split(" AND ")
    elif " OR " in query:
        operator = "OR"
        query_terms = query.split(" OR ")
    else:
        query_terms = [query] # Single word or simple multi-word

    # Preprocess the query terms
    processed_terms = []
    for qt in query_terms:
        processed_terms.extend(preprocess_text(qt))

    if not processed_terms:
        return set()

    # Retrieve sets of Document IDs for each term
    doc_sets = []
    for term in processed_terms:
        if term in inverted_index:
            doc_sets.append(set(inverted_index[term].keys()))
        else:
            doc_sets.append(set())

    # Apply Boolean Logic
    if not doc_sets:
        return set()

    result_docs = doc_sets[0]
    for i in range(1, len(doc_sets)):
        if operator == "AND":
            result_docs = result_docs.intersection(doc_sets[i])
        else: # OR
            result_docs = result_docs.union(doc_sets[i])

    return result_docs, processed_terms
```

CELL 6: TF-IDF Ranking

```
def rank_documents(result_doc_ids, query_terms):
    doc_scores = defaultdict(float)

    for term in query_terms:
        if term not in inverted_index:
            continue

        # Calculate IDF for the term
        # IDF = log(Total Documents / Number of docs containing the term)
        df_t = document_frequency[term]
        idf = math.log(Total_Documents / (float(df_t) + 1))

        for doc_id in result_doc_ids:
            # Check if this document contains the term
            if doc_id in inverted_index[term]:
                tf = inverted_index[term][doc_id]
                # Log-normalized term frequency is often better: 1 + log(tf)
                tf_normalized = 1 + math.log(tf)

                # Add to total document score
                doc_scores[doc_id] += tf_normalized * idf

    # Sort documents by score in descending order
    ranked_docs = sorted(doc_scores.items(), key=lambda item: item[1], reverse=True)
    return ranked_docs
```

CELL 7: Search Execution and Output Formatting

```
def search(query, top_k=10):
    print(f"\n--- Searching for: '{query}' ---")

    # 1. Process Boolean Query
    result_docs, processed_terms = get_boolean_results(query)

    if not result_docs:
        print("No matching jobs found.")
        return

    # 2. Rank the Results
    ranked_results = rank_documents(result_docs, processed_terms)

    # 3. Display Top K Results
    print(f"Found {len(ranked_results)} results. Showing top {top_k}:\n")

    for i, (doc_id, score) in enumerate(ranked_results[:top_k]):
        # Fetch actual row from dataframe
        job_data = df[df['job_id'] == doc_id].iloc[0]

        title = job_data['title']
        description = job_data['description']

        # Create a short snippet (approx 150 chars)
        snippet = description[:150].replace('\n', ' ') + "..."

        print(f"Rank {i+1} (Score: {score:.4f})")
        print(f"Title:   {title}")
        print(f"Snippet: {snippet}")
        print("-" * 60)
```

CELL 8: Testing Requirements

```
# Test 1: Single Word Query
search("Python")

# Test 2: Boolean AND Multi-word Query
search("Manager AND Remote")

# Test 3: Boolean OR Query
search("Developer OR Engineer")
```

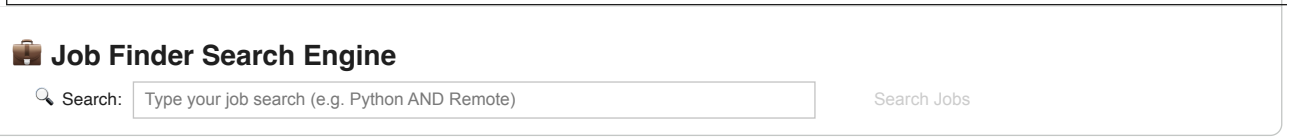
```
--- Searching for: 'Python' ---
Found 120 results. Showing top 10:

Rank 1 (Score: 9.7108)
Title:   Snowflake Developer
Snippet: Hi Everyone,***** W2 Contract Only*****100% Closure positionNote: we can accept all visa types and sur
------------------------------------------------------------
Rank 2 (Score: 8.8804)
Title:   Sr Python Developer
```

```
Snippet: We are seeking a highly skilled and motivated Senior Python Developer to join our team. The ideal cand
------------------------------------------------------------
Rank 3 (Score: 7.8098)
Title:   O9 Supply Chain Management Consultant
Snippet: O9 Consultant Sunnyvale, CA / Austin TXLong Term Contract / Full TimeDay 1 OnsiteTech Stack: O9 SCM, Py
------------------------------------------------------------
Rank 4 (Score: 7.8098)
Title:   Business Process Consultant - Strategic Portfolio Management (SPM)
Snippet: Principal Field Solution Architect - Azure AI - Digital Velocity The Principal Architect is responsibl
------------------------------------------------------------
Rank 5 (Score: 7.8098)
Title:   Hardware Design Engineer
Snippet: Pay Range: ($60/hr) the pay rate may differ depending on your skills, education, experience, and other
------------------------------------------------------------
Rank 6 (Score: 7.8098)
Title:   Associate - Applications Support
Snippet: Job Description  As an Application Support professional, in our Consumer and Community Banking team, yo
------------------------------------------------------------
Rank 7 (Score: 6.3009)
Title:   Knowledge Digitalization Engineer
Snippet: Location: Plymouth, MI  Hybrid working scheme: 3 days in office, 2 home office.  Travel 10%  About SKF
------------------------------------------------------------
Rank 8 (Score: 6.3009)
Title:   Senior Data Engineer (Property Reinsurance/ Speciality Reinsurance) - 100% Remote
Snippet: Our client is seeking talented and intellectually curious data engineers with a passion for the “R” in
------------------------------------------------------------
Rank 9 (Score: 6.3009)
Title:   Cloud Application Engineer
Snippet: About Trustwave  Trustwave is a leading cybersecurity and managed security services provider focused on
------------------------------------------------------------
Rank 10 (Score: 6.3009)
Title:   Sr Data Engineer with Kafka
Snippet: Data Engineer with Kafka (W2 Only)💯% Remote Min 10 to12+ strong development experience neededVery stro
------------------------------------------------------------

--- Searching for: 'Manager AND Remote' ---
Found 149 results. Showing top 10:

Rank 1 (Score: 10.3682)
Title:   Remote Behavioral Health Case Manager
Snippet: Piper Companies is currently seeking a Remote Behavioral Health Case Manager for a remote opportunity w
------------------------------------------------------------
Rank 2 (Score: 8.9866)
Title:   Regional Marketing Manager USCAN
Snippet: This inclusive employer is a member of myGwork – the largest global platform for the LGBTQ+ business co
------------------------------------------------------------
Rank 3 (Score: 8.5177)
```

CELL 9: Graphical User Interface (GUI) in Colab

```
import ipywidgets as widgets
from IPython.display import display, HTML

# Create UI Elements
search_box = widgets.Text(
    value='',
    placeholder='Type your job search (e.g. Python AND Remote)',
    description='🔍 Search:',
    layout=widgets.Layout(width='600px')
)

search_button = widgets.Button(
    description='Search Jobs',
    button_style='info',
    tooltip='Click to search',
    icon='search'
)

output_area = widgets.Output()

def on_search_button_clicked(b):
    with output_area:
        output_area.clear_output()
        query = search_box.value

        if not query:
            print("Please enter a search term.")
            return

        result_docs, processed_terms = get_boolean_results(query)

        if not result_docs:
            display(HTML("<h3 style='color:red;'>No matching jobs found.</h3>"))
            return

        ranked_results = rank_documents(result_docs, processed_terms)
```

```
        ranked_results = rank_documents(result_docs, processed_terms)

        # Build HTML Output to look like a modern search engine
        html_content = f"<h3>Found {len(ranked_results)} results for '<i>{query}</i>'</h3>"

        for i, (doc_id, score) in enumerate(ranked_results[:10]):
            job_data = df[df['job_id'] == doc_id].iloc[0]
            title = job_data['title']
            snippet = job_data['description'][:200].replace('\n', ' ') + "..."

            # Highlight keywords (Bonus requirement)
            for term in processed_terms:
                snippet = re.sub(f'(?i)({term})', r'<mark style="background-color: yellow;">\1</mark>', snippet)
                title = re.sub(f'(?i)({term})', r'<mark style="background-color: yellow;">\1</mark>', title)

            html_content += f"""
            <div style='margin-bottom: 20px; padding: 10px; border: 1px solid #ddd; border-radius: 8px;'>
                <div style='color: #1a0dab; font-size: 18px; font-weight: bold;'>{title}</div>
                <div style='color: #006621; font-size: 14px;'>Score: {score:.4f}</div>
                <div style='color: #545454; font-size: 14px; margin-top: 5px;'>{snippet}</div>
            </div>
            """
        display(HTML(html_content))

search_button.on_click(on_search_button_clicked)

# Display the UI
display(HTML("<h2>💼 Job Finder Search Engine</h2>"))
display(widgets.HBox([search_box, search_button]))
display(output_area)
```



CELL 9A: Save Search Engine Data for the App

```
import joblib

print("="*80)
print("SAVING ASSETS FOR WEB APP")
print("="*80)

# Convert defaultdicts to standard dicts for safe saving
engine_data = {
    "inverted_index": {k: dict(v) for k, v in inverted_index.items()},
    "document_frequency": dict(document_frequency),
    "Total_Documents": Total_Documents,
    "df": df
}

joblib.dump(engine_data, 'search_engine_data.pkl')
print("✅ Saved search engine assets as 'search_engine_data.pkl'. Ready for App.")
```

```
================================================================================
SAVING ASSETS FOR WEB APP
================================================================================
```

```
%%writefile app.py
import streamlit as st
import pandas as pd
import joblib
import math
import re
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

# Ensure NLTK data is available in the background script
nltk.download('punkt', quiet=True)
nltk.download('stopwords', quiet=True)
nltk.download('punkt_tab', quiet=True)
stop_words = set(stopwords.words('english'))

# Load Data
data = joblib.load('search_engine_data.pkl')
inverted_index = data['inverted_index']
document_frequency = data['document_frequency']
Total_Documents = data['Total_Documents']
df = data['df']
```

```
# Helper Functions
def preprocess_text(text):
    if pd.isna(text): return []
    text = str(text).lower()
    text = re.sub(r'[^a-z0-9\s]', '', text)
    return [word for word in word_tokenize(text) if word not in stop_words]

def get_boolean_results(query):
    operator = "AND" if " AND " in query else "OR"
    query_terms = query.split(f" {operator} ") if f" {operator} " in query else [query]

    processed_terms = []
    for qt in query_terms:
        processed_terms.extend(preprocess_text(qt))
    if not processed_terms:
        return set(), []

    doc_sets = [set(inverted_index.get(term, {}).keys()) for term in processed_terms]
    if not doc_sets:
        return set(), processed_terms

    result_docs = doc_sets[0]
    for i in range(1, len(doc_sets)):
        if operator == "AND":
            result_docs = result_docs.intersection(doc_sets[i])
        else:
            result_docs = result_docs.union(doc_sets[i])
    return result_docs, processed_terms

def rank_documents(result_doc_ids, query_terms):
    doc_scores = {}
    for term in query_terms:
        if term not in inverted_index: continue
        idf = math.log(Total_Documents / (float(document_frequency.get(term, 0)) + 1))
        for doc_id in result_doc_ids:
            if doc_id in inverted_index[term]:
                tf = inverted_index[term][doc_id]
                doc_scores[doc_id] = doc_scores.get(doc_id, 0) + (1 + math.log(tf)) * idf
    return sorted(doc_scores.items(), key=lambda x: x[1], reverse=True)

# Streamlit UI
st.set_page_config(page_title="JobFinder", page_icon="💼", layout="wide")
st.title("💼 JobFinder Search Engine")
st.markdown("Search millions of job listings using Boolean logic (e.g., `Python AND Remote`).")

query = st.text_input("🔍 Enter Search Query:")

if st.button("Search Jobs") or query:
    if not query:
        st.warning("Please enter a search term.")
    else:
        result_docs, processed_terms = get_boolean_results(query)

        if not result_docs:
            st.error(f"No matching jobs found for: **{query}**")
        else:
            ranked_results = rank_documents(result_docs, processed_terms)
            st.success(f"About {len(ranked_results)} results found.")
            st.divider()

            for doc_id, score in ranked_results[:10]: # Display top 10
                job_data = df[df['job_id'] == doc_id].iloc[0]
                title = str(job_data['title'])
                snippet = str(job_data['description'])[:200].replace('\n', ' ') + "..."

                st.subheader(title)
                st.caption(f"Relevance Score: {score:.4f}")
                st.write(snippet)
                st.markdown("---")
```

CELL 9C: Launch App via Pinggy (No Password)

```
import time

# 1. Install Streamlit
!pip install -q streamlit

print("------------------------------------------------------------")
print("🚀 STARTING THE APP... PLEASE WAIT 15 SECONDS")
print("------------------------------------------------------------")
```

```
# 2. Run Streamlit in the background with CORS disabled
get_ipython().system_raw('streamlit run app.py --server.enableCORS=false --server.enableXsrfProtection=false &')

# 3. Create the Tunnel using Pinggy
get_ipython().system_raw('ssh -o StrictHostKeyChecking=no -p 443 -R0:localhost:8501 a.pinggy.io > tunnel_url.txt

# 4. Wait for the URL to generate
time.sleep(15)

# 5. Extract and Print the URL
try:
    with open('tunnel_url.txt', 'r') as f:
        log_content = f.read()
        import re
        url_match = re.search(r'https://[\w-]+\.pinggy\.link', log_content)
        if url_match:
            public_url = url_match.group(0)
            print(f"\n✅ YOUR APP IS LIVE! CLICK HERE:\n{public_url}")
            print("\n(No password required. If it doesn't load instantly, refresh the page)")
        else:
            print("❌ Could not generate link. Retrying...")
            print(log_content)
except Exception as e:
    print(f"Error: {e}")

# Keep cell running
while True:
    time.sleep(1)
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9.2/9.2 MB 73.3 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 11.3/11.3 MB 78.3 MB/s eta 0:00:00
------------------------------------------------------------
🚀 STARTING THE APP... PLEASE WAIT 15 SECONDS
------------------------------------------------------------
❌ Could not generate link. Retrying...
Pseudo-terminal will not be allocated because stdin is not a terminal.
Warning: Permanently added '[a.pinggy.io]:443' (RSA) to the list of known hosts.
Allocated port 4 for remote forward to localhost:8501
You are not authenticated.
Your tunnel will expire in 60 minutes. Upgrade to Pinggy Pro to get unrestricted tunnels. https://dashboard.pingg
http://aqbxy-35-187-233-171.run.pinggy-free.link
https://aqbxy-35-187-233-171.run.pinggy-free.link
```

GitHub Repo:

Job Listing Search Engine Data Mining and Warehouse Lab project CSE 426